Java String and immutability

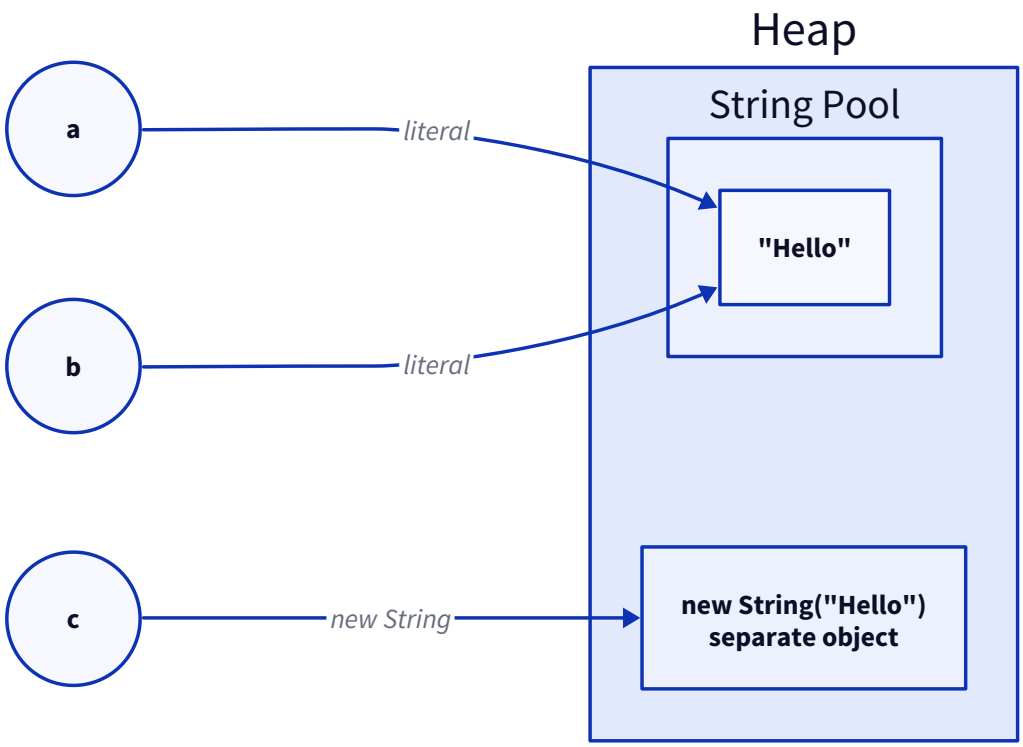
How String works, why it is immutable, how the String Pool and `==` behave, and where it hurts performance. Definitions stay tight; the optimisation and experience questions are how I would talk through them.

1. What is a String, why is it a class and not a primitive, and why is it so common?

A **String is an object that holds a sequence of characters**, backed internally by a `byte[]` (a `char[]` before Java 9). It is a **class, not a primitive**, because text needs behaviour, not just storage: methods like `length()`, `substring()`, `equals()`, and comparison. A primitive is a raw value with no methods, which cannot carry that. It is the most used class simply because **almost every program handles text**: input, output, keys, messages, so it earns first-class library support.

2. What is the difference between `String s = "Hello"` and `new String("Hello")`, and what happens internally?

A **literal** (`"Hello"`) goes through the **String Pool**: the JVM checks whether that value already exists in the pool and reuses it if so, so two identical literals point to the **same object**. `new String("Hello")` forces a **brand new object on the heap** every time, even if the same value is already in the pool. So the literal is the memory-friendly form, and `new String()` is almost always wasteful; the only time you would use it is if you deliberately want a distinct object reference.



Here `a` and `b` both point at the pooled `"Hello"`, so `a == b` is true, while `c` is a separate heap object, so `a == c` is false even though the text is equal.

3. Why is String declared `final`, and can it be inherited?

`String` is `final`, so it **cannot be inherited or subclassed**. That is deliberate: making it final guarantees **no one can override its methods to break immutability or the security assumptions** the JVM and libraries rely on. If you could subclass String and make it mutable, pooling and safe sharing would fall apart. So the answer to "can it be inherited" is no, by design.

4. What does immutability mean, and why are Strings immutable?

Immutable means the object cannot change after it is created: every operation that looks like a change (`concat`, `replace`, `toUpperCase`) returns a **new String** and leaves the original untouched. Strings are immutable for several reasons that are also the **benefits**:

- **Safe sharing and pooling**: because the value never changes, the pool can hand the same object to many references without risk.
- **Thread safety for free**: an immutable object needs no locking; many threads can read it safely.
- **Safe as map keys**: a String's hash cannot change under it, so it will not get lost in a `HashMap`.
- **Security**: a String passed to a file path or a connection cannot be altered after a security check.

5. If Strings are immutable, how does `concat()` appear to modify them?

It does not modify the original; it **returns a new String** with the combined value. The illusion comes from reassigning the variable: `s = s.concat("x")` makes `s` point at the new object, so it looks like `s` changed, but the old String is untouched (and now eligible for garbage collection if nothing else references it). Every "mutating" String method works this way.

6. If Strings are immutable, why can they still be garbage collected?

Immutability is about the **value not changing**, not about the object living forever. A String is still a normal heap object, so once **nothing references it**, it is collected like anything else. The one nuance is **pooled** Strings: while a literal is referenced by the pool it stays around, but ordinary (non-interned) Strings are collected as soon as they go out of scope.

7. Why isn't every immutable object pooled like String?

Because pooling only pays off when values are **frequently repeated and cheap to compare**, which is true for Strings and small boxed integers, but not in general. A pool costs memory and lookup time, and for arbitrary objects the **duplication rate is low**, so you would pay the pool overhead for little reuse. Java pools Strings (and caches small `Integer` / `Long` values) precisely because those repeat constantly; there is no benefit in pooling every object.

8. What is the String Constant Pool, why was it introduced, and where is it stored?

The **String Constant Pool (SCP)** is a special area where the JVM **stores one copy of each distinct String literal** and reuses it. It was introduced to **save memory**: text like `"true"` or a status code appears thousands of times, and pooling keeps a single copy instead of thousands. It reduces memory because identical literals **share one object** rather than each allocating their own. On where it lives: it moved from **PermGen to the**

regular heap in Java 7, which is why interned Strings can now be garbage collected and the pool can grow with the heap.

9. What is String deduplication?

String deduplication is a **garbage collector feature** (in G1 GC) that finds different String objects holding the **same characters** and points their internal `byte[]` at a single shared array. It is different from pooling: pooling shares the whole String object for literals, while deduplication runs at GC time on the **backing arrays** of any duplicate Strings, cutting memory without you calling `intern()`.

10. Heap v/s String Pool.

The **heap** is the general memory area where all objects live, including every String. The **String Pool** is a **region within the heap** reserved for unique String literals. So it is not a separate memory space; a pooled String is a heap object that the pool holds a reference to. The distinction that matters: a literal lands in the pool and is shared, while a `new String()` sits on the heap outside the pool as its own object.

11. Can the String Pool ever become a performance bottleneck?

Yes, if you **`intern()` aggressively**. Interning forces a String into the pool, and a pool bloated with millions of rarely-reused entries costs memory and slows the pool lookup, sometimes hurting more than it helps. So the mistake is **overusing `intern()`** hoping to save memory and instead creating a large, slow pool. Interning is worth it only for a **bounded set of values that repeat heavily**; otherwise leave Strings alone.

12. What is the difference between `==` and `equals()` for Strings?

`==` compares **references** (are these the same object), while `equals()` compares **content** (do they hold the same characters). This is the classic trap:

- `==` returns **true** when both point at the **same pooled literal** (two `"Hello"` literals), which fools people into thinking `==` works for text.
- `==` returns **false** when the values are equal but the objects differ (a literal v/s a `new String()`, or a String built at runtime).

`equals()` is overridden by String to compare the actual characters, which is why it is the correct way to compare String values. The rule: **always use `equals()` for String content**, never `==`.

13. Why is `equalsIgnoreCase()` needed, and why does locale matter in case conversion?

`equalsIgnoreCase()` compares content while **ignoring case** (`"HELLO"` equals `"hello"`), which you need for things like case-insensitive usernames. **Locale matters** because case conversion is language-dependent: the classic example is Turkish, where uppercasing `"i"` does not give `"I"`. So calling `toUpperCase()` without a locale can produce wrong results on some systems; for locale-independent logic you pass `Locale.ROOT`.

14. Why are String hash codes cached?

Because a String is **immutable, so its hash code can never change**, the class computes it **once on first use and stores it** in a field. After that, every `hashCode()` call (and every `HashMap` lookup) returns the cached value instead of rescanning the characters. This is a direct payoff of immutability, and it is why Strings are such efficient map keys.

15. Why isn't StringBuilder thread-safe, and how does it compare to StringBuffer?

`StringBuilder` is **not synchronised**, so two threads appending to the same instance can corrupt it. That was a deliberate choice: it is meant for the common case of **building a String on one thread**, where synchronisation would just be overhead. `StringBuffer` is the **synchronised, thread-safe** version, but it is slower because of the locking. So the guidance is `StringBuilder` by default, and `StringBuffer` only in the rare case a builder is genuinely shared across threads.

16. Why are passwords not stored as String, and why is `char[]` preferred?

Because a String is **immutable, you cannot wipe it**: once a password is in a String, it sits in memory (and possibly the pool) until garbage collection decides to reclaim it, so it can be read from a heap dump in the meantime. A `char[]` **is mutable**, so you can **overwrite it with zeros immediately after use**, shrinking the window where the password is exposed. That control over erasing the value is the whole reason `char[]` is preferred for secrets.

17. A production application is creating millions of temporary Strings. How would you optimise it?

The way I would approach it: first **confirm where the allocations come from** with a profiler or a heap allocation view, because you optimise the hot path, not a guess. The usual wins, in order: **replace String concatenation in loops with a single `StringBuilder`**, because `a = a + b` in a loop creates a new String every iteration; **cache or precompile things built repeatedly** (a compiled regex `Pattern` instead of calling `split()` each time); **avoid `new String()`** and needless `substring` / `toString` churn; and **reuse buffers** rather than allocating per request. If the same values recur heavily, bounded interning can help, but I would measure first. The theme is to cut per-request allocation on the paths that actually run millions of times, and leave the rest alone.

18. Have you used StringBuilder extensively, and in which scenarios?

Yes, mainly for **building a String piece by piece in a loop**: assembling a CSV line, a log message, a query, or any output where I am appending in iterations. That is exactly where `+` concatenation is a trap, because each `+` allocates a new String, so an N-iteration loop is quadratic in work, while a single `StringBuilder` appends into one growing buffer. For a one-off concatenation of a couple of values I still use `+` since the compiler handles it; the builder is for the repeated-append case. *(Adapt to your own examples.)*

19. How would you investigate a memory issue caused by excessive String retention or allocation?

I would start by telling apart the two cases: a **leak** (Strings retained and never freed) versus **churn** (lots of short-lived Strings). For allocation churn I would use a profiler or **Java Flight Recorder** to see the allocation hot spots and which call sites produce the most String garbage. For a retention leak I would take a **heap dump** and look at what is holding the Strings alive (a growing cache or collection is the usual culprit), following the reference chain to the root. Once I know whether it is churn or retention, the fix follows: reduce allocation on the hot path, or fix the collection that never releases entries. *(Adapt to a real investigation you did.)*

20. Have you hit a bug caused by using `==` instead of `equals()`?

It is one of the most common Java bugs, and the reason it is nasty is that it **works by accident in testing and fails in production**. With small string literals, `==` returns true because both sit in the pool, so a comparison looks correct in a unit test. Then in production the same String arrives **built at runtime** (from a request, a

database, a parse), it is a different object, `==` returns false, and the logic silently takes the wrong branch. The fix is always `equals()`, and it is why I flag any `==` on Strings in review. *(Adapt to your own example.)*

21. What String best practices does your team follow?

The ones I would call out: **compare with `equals()`**, never `==`; **`StringBuilder` for loop concatenation**; **compile regex `Pattern`s once** instead of calling `split()` or `matches()` repeatedly; **avoid `new String()`**; **pass a locale** to case conversion where correctness matters; and **never hold secrets in a String**. Most of these are caught in code review, and a few by static analysis. *(Adapt to your own team.)*

22. Common mistakes.

- **Comparing Strings with `==`** instead of `equals()`, which passes in tests and fails on runtime-built Strings.
- **Using `new String()` unnecessarily**, which allocates a redundant object and skips the pool.
- **Concatenating with `+` in a loop**, which creates a new String each iteration; use `StringBuilder`.
- **Calling `split()` repeatedly in a hot path**, which recompiles the regex every call; compile a `Pattern` once.
- **Storing passwords as String**, which you cannot wipe from memory; use `char[]`.
- **Assuming `trim()` removes all whitespace**; it only strips characters up to space (`U+0020`), so Unicode whitespace survives (use `strip()` from Java 11).
- **Ignoring locale in case conversion**, which breaks on locales like Turkish; pass `Locale.ROOT` for locale-independent logic.
- **Overusing `intern()`**, which can bloat the pool and slow lookups instead of saving memory.